

Bootloader 移植应用开发指南

EIC7700 AI 数字 SoC

Original Template / Rev 1.0

2024-02-29

声明

版权所有©2024，北京奕斯伟计算技术有限公司及其关联公司（以下简称“ESWIN”）。保留所有权利。

ESWIN 在此软件中保留所有知识产权和专有权利。未经 ESWIN 授权，不得发布、复制、分发、转载、修改、改编、翻译或制作此软件的衍生作品，无论是全部还是部分。

对于本文档的任何修改，ESWIN 无需承担通知义务且不代表 ESWIN 做出任何承诺。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证与承诺，无论明示或默示。

“ESWIN” logo 和其他 ESWIN 标识，为北京奕斯伟计算技术有限公司及（或）其母公司所有的商标。本文档提及的其他所有商标或商品名称，由其各自的所有权人所有。

产品安装中可能包含开源软件和/或自由软件的许可声明。

修订记录

文档版本	日期	说明
v1.3	2024/09/11	调整 5.1u-boot 外设清单至 1.2，添加 4.1 部分描述
v1.2	2024/09/09	修正章节调整引起的引用错误
v1.1	2024/08/14	添加图表目录，更新 5.3 分区表信息
v1.0	2024/07/29	添加 bootspi 写保护，与最新代码改动同步
v0.4	2024/02/29	Bootloader 移植应用开发指南

目 录

BOOTLOADER 移植应用开发指南	0
EIC7700 AI 数字 SoC	0
1. 概 述	1
1.1 U-BOOT 目录结构	1
1.2 UBOOT 外设支持清单	2
2. U-BOOT 下载及编译	2
2.1 U-BOOT 下载	2
2.2 交叉编译器下载	3
2.3 U-BOOT 编译	3
3. OPENSBI 下载及编译	3
3.1 OPENSBI 下载	3
3.2 OPENSBI 编译	3
3.3 BOOTCHAIN 制作	4
4. 移植 U-BOOT	4
4.1 基于 EIC7700_EVB_A2 新建 BOARD	4
4.2 基于参考 BOARD 直接修改	6
5. 附录	6
5.1 网络支持	6
5.2 创建分区表	7
5.3 烧写支持	7
5.4 BOOTSPI 写保护	12

表 目 录

表 1-1 U-Boot-2024.01 主要目录文件..... 1

图 目 录

图 5-1 EIC7700 board emmc 分区信息..... 7

1. 概述

EIC7700 Bootloader 主要由 U-Boot 和 opensbi 组成，用户在开发移植过程中只需要关注 U-Boot 即可，故本文主要描述 U-Boot 下载，编译，移植，opensbi 下载编译以及常用的 U-Boot 命令。

1.1 U-Boot 目录结构

表 1-1 U-Boot-2024.01 主要目录文件

名字	描述
api	与硬件无关的 API 函数
arch	与架构有关的代码
board	不同单板(开发板) 的定制代码
cmd	命令相关代码
common	通用代码
config	配置文件
disk	与磁盘分区相关代码
doc	文档
drivers	设备驱动代码
dts	设备树 Kconfig 和 Makefile
env	环境变量相关代码
example	示例代码
fs	文件系统实现代码
include	头文件数
lib	库文件
Licenses	许可证相关文件
net	网络相关文件
post	上电自检程序
scripts	脚本文件
test	测试代码
tools	工具文件夹
.config	配置文件，编译后生成
.gitignore	git 工具相关文件
.mailmap	邮件列表

config.mk	某个 Makefile 会调用此文件
Kbuild	用于生成一些和汇编有关的文件
Kconfig	图形配置界面描述文件
MAINTAINERS	维护者联系方式文件
MAKEALL	一个 shell 脚本，帮助生成 uboot 的
Makefile	主 Makefile
README	相当于帮助文档。

1.2 uboot 外设支持清单

EIC7700 uboot 支持以下外设：

- UART
- I2C
- GPIO
- SPI
- SD
- EMMC
- SATA
- ETH
- PCIE
- USB-HOST
- USB(fastboot)
- HDMI

2. U-Boot 下载及编译

2.1 U-Boot 下载

用户可以从 github 下载 ESWIN 提供的 U-Boot 源码.

```
git clone https://github.com/eswincomputing/u-boot.git --branch u-boot-2024.01-EIC7X
Cloning into 'u-boot'...
remote: Enumerating objects: 1002538,  done.
remote: Counting objects: 100% (5675/5675),  done.
remote: Compressing objects: 100% (3094/3094),  done.
remote: Total 1002538 (delta 2622),  reused 5274 (delta 2561),  pack-reused 996863
Receiving objects: 100% (1002538/1002538),  230.86 MiB | 901.00 KiB/s,  done.
Resolving deltas: 100% (832788/832788),  done.
```


2.2 交叉编译器下载

```
//下载交叉编译器
wget https://github.com/riscv-collab/riscv-gnu-toolchain/releases/download/2024.04.12/riscv64-glibc-ubuntu-22.04-gcc-nightly-2024.04.12-nightly.tar.gz
sudo tar -xvf riscv64-glibc-ubuntu-22.04-gcc-nightly-2024.04.12-nightly.tar.gz -C /opt

//配置环境变量
export PATH="/opt/riscv/bin:$PATH"
export ARCH=riscv
export CROSS_COMPILE=riscv64-unknown-linux-gnu-
```

注：操作系统要求-ubuntu 22.04

2.3 U-Boot 编译

```
//设置 U-Boot 的默认配置文件
make eic7700-evb_a2_defconfig
//编译 U-Boot 项目工程，编译成功后，会在该 u-boot 目录下生成 u-boot.bin/u-boot.dtb 镜像文件
make -j21
//清除编译生成的目录和文件
make clean
```

3. opensbi 下载及编译

3.1 opensbi 下载

```
git clone https://github.com/eswincomputing/opensbi.git --branch opensbi-1.0-EIC7X
Cloning into 'opensbi'...
remote: Enumerating objects: 7810, done.
remote: Counting objects: 100% (7810/7810), done.
remote: Compressing objects: 100% (2703/2703), done.
remote: Total 7810 (delta 4767), reused 7810 (delta 4767), pack-reused 0
Receiving objects: 100% (7810/7810), 1.63 MiB | 2.32 MiB/s, done.
Resolving deltas: 100% (4767/4767), done.
```

3.2 opensbi 编译

配置环境变量可参考 [2.2 交叉编译器下载](#)。

```
//编译 opensbi 项目工程，编译成功后，会在该 opensbi/build/platform/eswin/eic770x/firmware/目录下生成
fw_payload.bin 镜像文件

make -j21 PLATFORM=eswin/eic770x FW_PAYLOAD_PATH=$UBOOT_PATH/u-boot.bin
FW_FDT_PATH=$UBOOT_PATH/u-boot.dtb CHIPLET="BR2_CHIPLLET_1"
CHIPLET_DIE_AVAILABLE="BR2_CHIPLLET_1_DIE0_AVAILABLE" MEM_MODE="BR2_MEMMODE_FLAT"
PLATFORM_CLUSTER_X_CORE="BR2_CLUSTER_4_CORE" PLATFORM_RISCV_ISA=rv64imafdc_zicsr_zifencei

//清除编译生成的目录和文件
make clean
```

3.3 bootchain 制作

编译生成 fw_payload.bin 镜像文件后可参考《镜像制作和 EsNsign 用户手册.docx》制作 bootchain 文件。

4. 移植 U-Boot

4.1 基于 eic7700_evb_a2 新建 board

本节介绍如何完全新建 test_board，若用户不想使用 eic7700_evb_a2，需要新建自己的 board，可参考本节内容创建属于自己的 test_board。

4.1.1 添加 board 图形配置选项

在 Kconfig 中添加新建 board 选项，以便于图形化配置，修改 arch/riscv/Kconfig

```
config TARGET_TEST_BOARD          // 添加新 board config 选项
    bool "Support Eswin EIC7700 test Board"

.....

source "board/eswin/test_board/Kconfig" //添加 board 初始化代码 Kconfig
```

4.1.2 添加 board 默认配置文件

在 configs 目录下创建默认配置文件，复制 eic7700_evb_a2_defconfig

```
cd configs
cp eic7700_evb_a2_defconfig test_board_defconfig
```

然后根据实际的硬件设计对部分 CONFIG 内容进行修改，常见的修改点：

```
CONFIG_DEFAULT_DEVICE_TREE=" eic7700_evb_a2"    // 修改为"test_board"
CONFIG_TARGET_ESWIN_EIC7700=y                  // 修改为 CONFIG_TARGET_TEST_BOARD
CONFIG_ARCH_RV64I=y
```

注：

- CONFIG_DEFAULT_DEVICE_TREE：指定 dts 的文件名，编译 uboot 时会同时编译 dts，编译系统依据此参数到 arch/riscv/dts 目录寻找同名带.dts 后缀的文件，并生成.dtb 文件。
- CONFIG_TARGET_ESWIN_EIC7700：target 名，可自定义，但需与 4.1.1 中新增的 board config 选项同名

4.1.3 添加 dts 文件

在 arch/riscv/dts 目录下创建新 board 的 dts 文件，可直接复制 eic7700_evb_a2.dts 后再进行修改：

```
cd arch/riscv/dts
cp eic7700_evb_a2.dts test_board.dts
```

对新建的 test_board.dts 可根据具体 board 的硬件设计，添加或删除部分设备描述。

修改 arch/riscv/dts/Makefile，添加新 board 的编译：

```
dtb-$(CONFIG_TARGET_TEST_BOARD) += test_board.dtb
//名字需与 defconfig 中 CONFIG_DEFAULT_DEVICE_TREE 的值一致
```

4.1.4 添加 board 对应的板级文件夹

U-Boot 中每个 board 都有一个对应的文件夹存放板级文件，该目录中可以放诸如 board 的初始化文件等，直接复制后再进行修改：

```
cd board/eswin
cp -r eic7700_evb test_board
```

对 test_board 目录中文件再做相关修改

修改文件名

将复制后的 test_board 下文件重新命名：

```
cd test_board
mv eic7700.c test_board.c
```

修改 Makefile

文件重新命名后，Makefile 也要做同步修改：

```
obj-y += test_board.o
```

修改 Kconfig

```
if TARGET_ESWIN_EIC7700           //修改为 TARGET_TEST_BOARD，需与 defconfig 中
config SYS_BOARD
    default "eic7700_evb"         //修改为 "test_board"

config SYS_VENDOR
    default "eswin"

config SYS_CPU
    default "eic700"

config SYS_CONFIG_NAME
    default "eswin_eic7700_evb"   //修改为 "test_board"，与添加 board 头文件文件名相同
```

添加 board 对应的头文件

在 include/configs 目录下创建新 board 的头文件，直接复制后再进行修改：

```
cd include/configs
cp eswin_eic7700_evb.h test_board.h
```

该头文件大多是以"CONFIG_"开头的宏定义为主的 uboot 配置文件，包含了 bootcmd，env，malloc len 和 bootm len 等信息。

4.1.5 编译 uboot

前面部分已经完成了新 board 的创建工作，现在可以直接编译生成 u-boot.bin，在 u-boot 目录下执行如下命令，编译通过后将会在 u-boot 目录下生成 u-boot.bin 和 u-boot.dtb。交叉编译下载及环境变量配置可参考 [2.2 交叉编译器下载](#)。

```
make test_board_defconfig
make -j21
```

4.2 基于参考 board 直接修改

相对于新建 board，直接修改参考 board(eic7700-evb_a2)要更加的简单便捷。

4.2.1 修改文件

如 3.1 节新建 board 中介绍，直接修改 eic7700-evb_a2 的 board 文件，可从下目录及文件入手：

1. configs/eic7700-evb_a2_defconfig
包含 CONFIG_XXX 相关的功能和特性开关。
2. arch/riscv/dts/eic7700-evb_a2.dts, eic770x.dtsi
该目录为 board 对应的 dts 文件，与 kernel 中 dts 功能相同。
3. board/eswin/eic7700-evb/
包含 eic7700 board 初始化。
4. include/configs/eswin_eic7700-evb.h
board 配置文件，包括 malloc len, bootm len, env, bootcmd 等。

4.2.2 编译

在 u-boot 目录，执行如下命令，编译成功后，会在该目录下生成 u-boot.bin 和 u-boot.dtb。交叉编译下载及环境变量配置可参考 [2.2 交叉编译器下载](#)。

```
make eic7700-evb_a2_defconfig
make -j21
```

5. 附录

5.1 网络支持

EIC7700 参考 board 的 U-Boot 支持网络功能，可以在 cmd 模式下通过网络给开发板传输镜像，步骤如下所示：

- 1) 在本地或者 linux 服务器建立 tftp server
- 2) 在 u-boot 启动过程中按键，进入 cmd 模式
- 3) 输入命令，下载镜像到开发板内存中

```
dhcp 0x100000000 10.10.192.202:xxx.bin
```

其中，10.10.192.202 为服务器端 tftp serverip，使用时需根据具体的 Server IP 修改，0x100000000 为开发板内存地址，该命令作用是将开发板连入网络后，通过 server 端 tftp server，将 bin 文件传输到开发板内存地址 0x100000000。

5.2 创建分区表

EIC7700 board 支持对 EMMC/SD 进行分区，目前 EIC7700 board 支持的分区分布如下：

Partition	Name	size/type
1	boot	512MiB/ext4(start=1MiB)
2	swap	4096MiB/swap
3	root	-/ext4

图 5-1 EIC7700 board emmc 分区信息

通过执行以下命令，可实现对 emmc 进行分区：

1. 用户设定

```
=>env edit partitions
edit:name=boot,start=1MiB,size=512MiB,type=C12A7328-F81F-11D2-BA4B-
00A0C93EC93B;name=swap,size=4096MiB,type=0657FD6D-A4AB-43C4-84E5-
0933C84B4F4F,uuid=5ebcaaf0-e098-43b9-beef-1f8deedd135e;name=root,uuid=80a5a8e9-
c744-491a-93c1-4f4194fd690a,type=0FC63DAF-8483-4772-8E79-3D69D8477DE4,size=-;

// emmc 创建分区表

=> gpt write mmc 0 $partitions

Writing GPT: success!

//SD 卡创建分区表

=> gpt write mmc 1 $partitions
```

2. 使用默认分区

```
=>run gpt_partition

Writing GPT: success!
```

5.3 烧写支持

EIC7700 U-Boot 增加了 bootchain，boot 文件系统和 root 文件系统烧录功能，用户可通过特定的命令将对应的固件写入 flash/emmc，从而更新对应的固件。

在 uboot 命令行界面输入 es_burn, 可查看 cmd 用法, 具体如下:

```
=> es_burn

es_burn - ESWIN burn tool

Usage:
es_burn

es_burn write addr flash_stg      - write binary file from memory at `addr` to mtd

es_burn erase fw_type flash_stg - erase bootloader binary file who type is 'fw_type' from 'flash_stg'

es_burn wboot addr len flash_stg      - write bootmenu mode boot filesystem binary file from memory at `addr` to
mtd 'flash_stg'

es_burn wroot addr len flash_stg      - write bootmenu mode root filesystem binary file from memory at `addr` to
mtd 'flash_stg'
```

注:

- a. es_burn write addr flash_stg
功能: 用于烧写 bootchain 到目标存储设备
参数说明:
addr: bootchain 存放的内存地址
flash_stg: 烧写 bootchain 的目标存储设备。flash_stg 仅支持 flash, 即指定 bootchain 写入 flash。
- b. a. es_burn erase fw_type flash_stg
功能: 用于擦除写入存储设备中的 bootchain
参数说明:
fw_type: bootchain 的类型: all
flash_stg: 从目标存储设备中擦除 bootchain, 仅支持 flash, 即指定擦除 flash 中 bootchain。
- c. es_burn wboot len flash_stg
功能: 用于烧写 boot 文件系统到目标存储设备
参数说明:
addr: boot 文件系统存放的内存地址
len: boot 文件系统十六进制大小
flash_stg: boot 文件系统写入的存储设备类型, 仅支持 emmc, 指将 boot.ext4 写入 emmc
- d. es_burn wroot addr len flash_stg
功能: 用于烧写 root 文件系统
参数说明:
addr: 文件系统存放的内存地址
len: 文件系统十六进制大小
flash_stg: root 文件系统写入的存储设备类型, 仅支持 emmc, 指将 rootfs 写入 emmc

5.3.1 烧写 bootchain

如下所示, 用户可通过网络将 bootchain 镜像(bootloader_ddr5_secboot.bin)下载到 EIC7700 开发板

的内存中，然后执行 es_burn 命令将 bootchain 镜像写入 flash 中，详细分区信息可参考[创建分区表](#)。

```
=> dhcp 0x100000000 10.10.192.202:bootloader_ddr5_secboot.bin // 下载 bootchian 镜像
ethernet@50400000 Waiting for PHY auto negotiation to complete..... done
BOOTP broadcast 1
BOOTP broadcast 2
BOOTP broadcast 3
DHCP client bound to address 10.10.207.45 (821 ms)
Using ethernet@50400000 device
TFTP from server 10.10.192.202; our IP address is 10.10.207.45
Filename 'bootloader_ddr5_secboot.bin'.
Load address: 0x100000000
Loading: #####
#####
#####
#####
#####
#####
1.9 MiB/s
done
Bytes transferred = 5005432 (4c6078 hex)
=> es_burn write 0x100000000 flash // 烧写 bootchian
SF: 224 bytes @ 0x0 Read: OK
FIRMWARE writing...
Erase progress: 100%:+++++
SF: 1910 bytes @ 0x800000 Erased: OK
Write progress: 100%:+++++
SF: 0x776 bytes @ 0x800000 Written: OK
DDR writing...
Erase progress: 100%:+++++
SF: 1062690 bytes @ 0x400000 Erased: OK
Write progress: 100%:+++++
SF: 0x103722 bytes @ 0x400000 Written: OK
BOOTLOADER writing...
Erase progress: 100%:+++++
SF: 3940024 bytes @ 0xa00000 Erased: OK
Write progress: 100%:+++++
SF: 0x3c1eb8 bytes @ 0xa00000 Written: OK
Erase progress: 100%:+++++
SF: 224 bytes @ 0x0 Erased: OK
Write progress: 100%:+++++
SF: 0xe0 bytes @ 0x0 Written: OK
bootloader write OK
```

注:

1.根据代码所示，命令“es_burn write addr flash_stg”中 addr 为 0x100000000，即通过 dhcp 将 bin 文件传输到开发板内存地址。

2.根据烧写镜像文件不同，es_burn 在执行过程中会提示当前写入镜像是 DDR，FIRMWARE 还是

BOOTLOADER 等，当提示 bootloader write ok，即表示烧写成功。

5.3.2 烧写 boot.ext4

如下所示，用户可通过网络将 boot.ext4 镜像下载到 EIC7700 开发板内存中，然后执行 es_burn 命令将 boot.ext4 镜像写入 emmc 对应的分区(partition boot)，详细分区信息可参考[创建分区表](#)。

```
=> dhcp 0x100000000 10.10.192.202:boot-evb2_ddr5.ext4
ethernet@50400000 Waiting for PHY auto negotiation to complete..... done
BOOTP broadcast 1
BOOTP broadcast 2
BOOTP broadcast 3
DHCP client bound to address 10.10.207.29 (1753 ms)
Using ethernet@50400000 device
TFTP from server 10.10.192.202; our IP address is 10.10.207.29
Filename ' boot-evb2_ddr5.ext4'.
Load address: 0x100000000
Loading:T #####
#####
#####
#####
#####
.....
#####
#####
#####
#####
#####
#####
#####
#####
6.7 MiB/s
done
Bytes transferred = 524288000 (1f400000 hex)
=> es_burn wboot 0x100000000 0x1f400000 emmc
Write progress: 100%:+++++
boot has been successfully writen in 0#boot
=>
```

注:

1. 根据代码所示，命令“es_burn wboot addr len flash_stg”中 addr 为 0x100000000，即通过 dhcp 将 boot-evb2_ddr5.ext4 传输到开发板内存地址；len 为 0x1f400000，即下载到开发板内存中的 boot-evb2_ddr5.ext4 大小。

2. 当提示 “boot has been successfully writen in 0#boot”，则表示 boot-evb2_ddr5.ext4 已经成功被写入 mmc 0# boot 中，boot-evb2_ddr5.ext4 烧写完成。

5.3.3 烧写文件系统

EIC7700 开发板默认使用的文件系统格式为 ext2/ext4。同样的，在烧写文件系统时，也需要将制作好的文件系统下载到开发板内存中，然后使用 es_burn 命令将文件系统写入 emmc 对应的分区(partition root)，详细分区信息可参考[创建分区表](#)。

```
=> dhcp 0x100000000 10.10.192.202: root-evb2_ddr5.ext4
ethernet@50400000 Waiting for PHY auto negotiation to complete..... done
BOOTP broadcast 1
DHCP client bound to address 10.10.207.29 (1750 ms)
Using ethernet@50400000 device
TFTP from server 10.10.192.202; our IP address is 10.10.207.29
Filename ' root-evb2_ddr5.ext4'.
Load address: 0x100000000
Loading:T#####
#####
#####
#####
.....
#####
#####
#####
6.6 MiB/s
done
Bytes transferred = 7516192768 (1c0000000 hex)
=> es_burn wroot 0x100000000 0x1c0000000 emmc
Write progress: 100%:+++++
root has been successfully written in 0#root
```

注:

- 1.根据代码所示，命令“es_burn wroot addr len flash_stg”中 addr 为 0x100000000，即通过 dhcp 将文件系统传输到开发板内存地址；len 为 0x1c0000000，即下载到开发板内存中的文件系统大小。
- 2.当提示“root has been successfully written in 0#root”，则表示文件系统已经成功被写入 mmc 0#root 中，文件系统烧写完成。

5.4 bootspi 写保护

由于 EIC7700 bootloader 存放于 bootspi flash 中，为了防止 bootloader 被破坏，在 uboot 中添加 bootspi 写保护命令，在 uboot cmd 界面输入 bootspi_wp，可查看 cmd 用法，具体如下：

```
=> bootspi_wp
Usage: bootspi_wp <0|1>
bootspi_wp - Enable or disable BootSPI write protection

Usage:
bootspi_wp <0|1> - 0 to disable, 1 to enable write protection
```